Project 3 – Propositional Logic and Naïve Bayes (300 pts)

In this project, you are allowed to use the following Python modules

- numpy
- matplotlib.pyplot
- os
- time
- itertools
- nltk
- re
- math

Task 1 PL Entailment Proving (100 pts)

In this task, you will implement the PL-resolution algorithm and the DPLL to prove propositional logic entailment. To simplify the parsing of PL sentences, you can assume that all the sentences in the knowledge base and the entailment's conclusion are disjuncts of literals. For example, a KB might be

$$KB = \{\text{"!A|B"}, \text{"!A|!B|C"}, \text{"A"}\}$$

, where "!" represents "NOT", "|" represents "OR", and each quoted string is a clause in the KB. In PL language, the above KB is:

$$(\neg A \vee B) \wedge (\neg A \vee \neg B \vee C) \wedge A$$

A query for this KB might be:

$$\alpha = \{A|B, C\}$$

, which is represented by PL language as

$$(A \vee B) \wedge C$$

To prove that $KB \vdash \alpha$, you will need to prove that $KB \wedge \neg\alpha$ is unsatisfiable.

Note that the literals in a disjunct clause can be in any order. For example, "!A|!B|C", "!B|C|!A", "C|!B|!A", and so on, are actually the same clause. Your program must properly handle the order wherever it is needed.

In the provided starter program PL_starter.ipynb, a few KBs and queries formatted like above examples are provided. You can first use them to verify the correctness of your functions. We also provide a function named kbq_gen() in the starter code, which can generate a random pair of KB and α. You should set its parameters to create different pairs to thoroughly test your program.

Submission Requirements

1. Your Python program. Name it PL_YourLastName_YourFirstName.ipynb

2. A report covering the following questions:
(1) How you designed your functions.
(2) Comparison between the two algorithms in terms of running time, completeness, and scalability. Please include plots and detailed analysis.

Task 2 Naïve Bayes Classifier (200 pts)

In this task, you will build and evaluate a **Multinomial Naive Bayes** (bag-of-words) model for classifying texts.

Dataset Requirements

You must build your models for at least two datasets:

- **Provided Dataset (see the 5 txt files)**
  - **Training set:** 400 sentences in total — 100 sentences from each discipline (physics, biology, world history, economics)
  - **Testing set:** 200 sentences (labeled)
- **Extra Dataset**
  - You must download a dataset from online resources such as Kaggle and UCI Machine Learning Repo.

Project Requirements

1. Preprocessing:
   - Tokenize (split) sentences appropriately.
   - Normalize text (e.g., converting to lowercase, punctuation removal, stop-words removal, and other preprocessing techniques that might help improve your model)
2. Model:
   - Implement a **Multinomial Naive Bayes classifier** using the training data.
   - Use **appropriate smoothing** (e.g., Laplace smoothing) to avoid zero probabilities for unseen words.
3. Prediction:
   - According to the model of bag of words, apply the trained model on the test set sentences to predict their underlying labels. Each sentence should be evaluated for its probability of being in each category. The category that has the highest probability will be your prediction.
   - You must properly handle possible numerical instability due to multiplying many small probability values (consider using logarithm)
4. Evaluation:

- Find the confusion matrix based on your testing results
- Analyze and **interpret the confusion matrix for each category**:
    - Identify which classes are the most often confused.
    - Provide example sentences of misclassification. Provide reasons why misclassification happened to these sentences.
- Evaluate the model accuracy under different pre-processing options and settings of model control parameters

Submission Requirements

1. Your python program. Name it NB_YourLastName_YourFirstName.py
   (note: it is OK to write and submit two programs, with each for one dataset)
2. Your extra dataset. If too large to upload, you must provide an effective weblink in your report.
3. Screenshots of the two confusion matrices printed by your program
4. A report covering the following items
    - Explanation of preprocessing and smoothing choices.
    - The confusion matrix on the test data.
    - Interpretation and discussion of the results.
    - Optimization efforts and results
    - Any interesting findings from your evaluation step (plots and analyses)